P1135R3
The C++20 Synchronization Library

Published Proposal, 2019-01-21

Authors:

Bryce Adelstein Lelbach (NVIDIA)

Olivier Giroux (NVIDIA)

JF Bastien (Apple)

Detlef Vollmann

Source:

GitHub

Issue Tracking:

GitHub

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

LWG

Table of Contents

§ 1. Introduction

This paper is the unification of a series of related C++20 proposals for introducing new synchronization and thread coordination facilities and enhancing existing ones:

— [P0514R4]: Efficient `atomic` waiting and semaphores.

— [P0666R2]: Latches and barriers.

— [P0995R1]: `atomic_flag::test` and lockfree integral types.

— [P1258R0]: Don't make C++ unimplementable for small CPUs.

§ 2. Changelog

Revision 0: Post Rapperswil 2018 changes from [P0514R4], [P0666R2], and [P0995R1] based on Rapperswil 2018 LEWG feedback.

— Refactored `basic_barrier` and `barrier` into one class with a default template parameter as suggested by LEWG at Rapperswil 2018.

— Refactored `basic_semaphore` and `counting_semaphore` into one class with a default template parameter as suggested by LEWG at Rapperswil 2018.

— Fixed `update` parameters in semaphore, latch, and barrier member functions to consistently default to 1 to resolve mistakes identified by LEWG at Rapperswil 2018.

Revision 1: Pre San Diego 2018 changes based on Rapperswil 2018 LEWG feedback and a June discussion on the LEWG and SG1 mailing lists.

— Added member function versions of `atomic_wait_*` and `atomic_notify_*`, for consistency. Refactored wording to accommodate this.

— Renamed the `atomic_flag` overloads of `atomic_wait` and `atomic_wait_explicit` to `atomic_flag_wait` and `atomic_flag_wait_explicit` for consistency and to leave the door open for future compatibility with C.

— Renamed `latch::arrive_and_wait` and `barrier::arrive_and_wait` to `latch::sync` and `barrier::sync`, because LEWG at Rapperswil 2018 expected these methods to be the common use case and prefers they have a short name.

— Renamed `latch::arrive` to `latch::count_down` to further separate and distinguish the `latch` and `barrier` interfaces.

— Removed `barrier::try_wait` to resolve concerns raised during LEWG discussion at Rapperswil 2018 regarding its "maybe consuming" nature.

— Required that `barrier::arrival_token`'s move constructor and move assignment operators are `noexcept` to resolve discussions in LEWG at Rapperswil 2018 regarding exceptions being thrown when using the split arrive and wait barrier interface.

— Made `counting_semaphore::acquire`, `counting_semaphore::try_acquire`, and `latch::wait` `noexcept`, because participants in the mailing list discussion preferred that synchronization operations not throw and that any resource acquisition failures be reported by throwing during construction of synchronization objects.

— Made `counting_semaphore`, `latch`, and `barrier`'s constructors non `constexpr` and allowed them to throw `system_error` if the object cannot be created, because participants in the mailing list discussion preferred that synchronization operations not throw and that any resource acquisition failures be reported by throwing during construction of synchronization objects.

— Clarified that `counting_semaphore::release`, `latch::count_down`, `latch::sync`, `barrier::wait`, `barrier::sync`, and `barrier::arrive_and_drop` throw nothing (but cannot be `noexcept`, because they have preconditions) to resolve discussions in LEWG at Rapperswil 2018 and on the mailing list.

Revision 2: San Diego 2018 changes to incorporate [P1258R0] and pre-meeting feedback.

— Made `barrier::wait` take its `arrival_token` parameter by rvalue reference.

— Made the `atomic_signed_lock_free` and `atomic_unsigned_lock_free` types optional for freestanding implementations, as per [P1258R0].

Revision 3: Pre Kona 2019 changes based on San Diego 2018 LEWG feedback.

— Renamed `latch::sync` and `barrier::sync` back to `latch::arrive_and_wait` and `barrier::arrive_and_wait`, because this name had the strongest consensus in LEWG at San Diego 2018.

— Removed `atomic_int_fast_wait_t` and `atomic_uint_fast_wait_t`, because LEWG at San Diego 2018 felt that the use case was uncommon and the types had high potential for misuse.

— Made `counting_semaphore::acquire` and `latch::wait` non `noexcept` again, because LEWG at San Diego 2018 desired `constexpr` constructors for new synchronization objects to allow synchronization during program initialization and to maintain consistency with existing synchronization objects like `mutex`.

— Made `counting_semaphore`, `latch`, and `barrier`'s constructors `constexpr` again, because LEWG at San Diego 2018 desired `constexpr` constructors for new synchronization objects to

allow synchronization during program initialization and to maintain consistency with existing synchronization objects like `mutex`.

— Clarified that `counting_semaphore::release`, `latch::count_down`, `latch::arrive_and_wait`, `barrier::wait`, `barrier::arrive_and_wait`, and `barrier::arrive_and_drop` may throw `system_error` exceptions, which is an implication of the constructors of said objects being `constexpr` because any underlying system errors must be reported on operations not during construction.

— Added missing `atomic<T>::wait` and `atomic<T>::notify_*` member functions to the class synopses for the `atomic<T>` integral, floating-point, and pointer specializations.

— Fixed `atomic<T>::notify_*` member functions to be non `const`.

§ 3. Wording

> Note: The following changes are relative to the post San Diego 2018 working draft of ISO/IEC 14882, ([N4791]).

> Note: The ❡ character is used to denote a placeholder number which shall be selected by the editor.

Add `<semaphore>`, `<latch>`, and `<barrier>` to Table 19 "C++ library headers" in **[headers]**.

Modify the header synopsis for `<atomic>` in **[atomics.syn]** as follows:

30.2 Header <atomic> synopsis **[atomics.syn]**

```
namespace std {
  // ...

  // 30.8, non-member functions
  // ...
  template<class T>
    void atomic_wait(const volatile atomic<T>*,
                     typename atomic<T>::value_type);
  template<class T>
    void atomic_wait(const atomic<T>*,
                     typename atomic<T>::value_type);
  template<class T>
    void atomic_wait_explicit(const volatile atomic<T>*,
                              typename atomic<T>::value_type,
                              memory_order);
  template<class T>
    void atomic_wait_explicit(const atomic<T>*,
                              typename atomic<T>::value_type,
                              memory_order);
  template<class T>
    void atomic_notify_one(volatile atomic<T>*);
  template<class T>
    void atomic_notify_one(atomic<T>*);
  void atomic_notify_one(volatile atomic_flag*);
  void atomic_notify_one(atomic_flag*);
  template<class T>
    void atomic_notify_all(volatile atomic<T>*);
  template<class T>
    void atomic_notify_all(atomic<T>*);
  void atomic_notify_all(volatile atomic_flag*);
  void atomic_notify_all(atomic_flag*);

  // 30.3, type aliases
  // ...

  using atomic_intptr_t       = atomic<intptr_t>;
  using atomic_uintptr_t      = atomic<uintptr_t>;
  using atomic_size_t         = atomic<size_t>;
  using atomic_ptrdiff_t      = atomic<ptrdiff_t>;
  using atomic_intmax_t       = atomic<intmax_t>;
  using atomic_uintmax_t      = atomic<uintmax_t>;
```

```
  using atomic_signed_lock_free   = see below;
  using atomic_unsigned_lock_free = see below;

  // ...

  // 30.9, flag type and operations
  struct atomic_flag;
  bool atomic_flag_test(volatile atomic_flag*) noexcept;
  bool atomic_flag_test(atomic_flag*) noexcept;
  bool atomic_flag_test_explicit(volatile atomic_flag*, memory_order) noex
  bool atomic_flag_test_explicit(atomic_flag*, memory_order) noexcept;
  bool atomic_flag_test_and_set(volatile atomic_flag*) noexcept;
  bool atomic_flag_test_and_set(atomic_flag*) noexcept;
  bool atomic_flag_test_and_set_explicit(volatile atomic_flag*, memory_orc
  bool atomic_flag_test_and_set_explicit(atomic_flag*, memory_order) noexc
  void atomic_flag_clear(volatile atomic_flag*) noexcept;
  void atomic_flag_clear(atomic_flag*) noexcept;
  void atomic_flag_clear_explicit(volatile atomic_flag*, memory_order) noe
  void atomic_flag_clear_explicit(atomic_flag*, memory_order) noexcept;
  void atomic_flag_wait(const volatile atomic_flag*, bool) noexcept;
  void atomic_flag_wait(const atomic_flag*, bool) noexcept;
  void atomic_flag_wait_explicit(const volatile atomic_flag*, bool, memory
  void atomic_flag_wait_explicit(const atomic_flag*, bool, memory_order) n
  void atomic_flag_notify_one(volatile atomic_flag*) noexcept;
  void atomic_flag_notify_one(atomic_flag*) noexcept;
  void atomic_flag_notify_all(volatile atomic_flag*) const noexcept;
  void atomic_flag_notify_all(atomic_flag*) const noexcept;
  #define ATOMIC_FLAG_INIT see below

  // 30.10, fences
  extern "C" void atomic_thread_fence(memory_order) noexcept;
  extern "C" void atomic_signal_fence(memory_order) noexcept;
}
```

Modify [**atomics.alias**] as follows:

30.3 Type aliases **[atomics.alias]**

1 The type aliases `atomic_intN_t`, `atomic_uintN_t`, `atomic_intptr_t`, and `atomic_uintptr_t` are defined if and only if `intN_t`, `uintN_t`, `intptr_t`, and `uintptr_t` are defined, respectively.

2 The type aliases `atomic_signed_lock_free` and `atomic_unsigned_lock_free` are defined to be specializations of `atomic` whose template arguments are integral types, respectively signed and unsigned, other than `bool`. In freestanding implementations (4.1), these aliases are optional. If an implementation provides a integral specialization of `atomic` other than `bool` for which `is_always_lock_free` is true, it shall define `atomic_signed_lock_free` and `atomic_unsigned_lock_free`. Otherwise, they shall not be defined. `is_always_lock_free` shall be `true` for `atomic_signed_lock_free` and `atomic_unsigned_lock_free`. An implementation which defines these type aliases should choose the integral specialization of `atomic` for which the atomic waiting and notifying operations are most efficient.

Note: The reference to "atomic waiting and notifying operations" in the above change should refer to the new **[atomic.wait]** subclause.

Add a new subclause after **[atomics.lockfree]**:

30.  Waiting and notifying **[atomics.wait]**

1 ***Atomic waiting and notifying operations*** provide a mechanism to wait for the value of an atomic object to change more efficiently than can be achieved with polling.

2 The following functions are ***atomic waiting operations***:

(2.1) — `atomic<T>::wait`.

(2.2) — `atomic_flag::wait`.

(2.3) — `atomic_wait` and `atomic_wait_explicit`.

3 The following functions are ***atomic notifying operations***:

(3.1) — `atomic<T>::notify_one` and `atomic<T>::notify_all`.

(3.2) — `atomic_flag::notify_one` and `atomic_flag::notify_all`.

(3.3) — `atomic_notify_one` and `atomic_notify_one_explicit`.

(3.4) — `atomic_flag_notify_one` and `atomic_flag_notify_one_explicit`.

(3.5) — `atomic_notify_all` and `atomic_notify_all_explicit`.

(3.6) — `atomic_flag_notify_all` and `atomic_flag_notify_all_explicit`.

4 Atomic waiting operations in this facility may block until they are unblocked by atomic notifying operations, according to each function's effects. [*Note:* Programs are not guaranteed to observe transient atomic values, an issue known as the A-B-A problem, resulting in continued blocking if a condition is only temporarily met. – *end note*]

Modify [**atomics.types.generic**] as follows:

30.7 Class template `atomic` **[atomics.types.generic]**

```
namespace std {
  template<class T> struct atomic {
    using value_type = T;
    static constexpr bool is_always_lock_free = implementation-defined;
    bool is_lock_free() const volatile noexcept;
    bool is_lock_free() const noexcept;
    void store(T, memory_order = memory_order::seq_cst) volatile noexcept;
    void store(T, memory_order = memory_order::seq_cst) noexcept;
    T load(memory_order = memory_order::seq_cst) const volatile noexcept;
    T load(memory_order = memory_order::seq_cst) const noexcept;
    operator T() const volatile noexcept;
    operator T() const noexcept;
    T exchange(T, memory_order = memory_order::seq_cst) volatile noexcept;
    T exchange(T, memory_order = memory_order::seq_cst) noexcept;
    bool compare_exchange_weak(T&, T, memory_order, memory_order) volatile
    bool compare_exchange_weak(T&, T, memory_order, memory_order) noexcept
    bool compare_exchange_strong(T&, T, memory_order, memory_order) volati
    bool compare_exchange_strong(T&, T, memory_order, memory_order) noexce
    bool compare_exchange_weak(T&, T, memory_order = memory_order::seq_cst
    bool compare_exchange_weak(T&, T, memory_order = memory_order::seq_cst
    bool compare_exchange_strong(T&, T, memory_order = memory_order::seq_c
    bool compare_exchange_strong(T&, T, memory_order = memory_order::seq_c
    void wait(T old, memory_order = memory_order::seq_cst) const volatile
    void wait(T old, memory_order = memory_order::seq_cst) const noexcept;
    void notify_one() volatile noexcept;
    void notify_one() noexcept;
    void notify_all() volatile noexcept;
    void notify_all() noexcept;
    atomic() noexcept = default;
    constexpr atomic(T) noexcept;
    atomic(const atomic&) = delete;
    atomic& operator=(const atomic&) = delete;
    atomic& operator=(const atomic&) volatile = delete;
    T operator=(T) volatile noexcept;
    T operator=(T) noexcept;
  };
}
```

Add the following to the end of [**atomics.types.operations**]:

```
void wait(T old, memory_order order = memory_order::seq_cst) const volatil
void wait(T old, memory_order order = memory_order::seq_cst) const noexcep
```

1 *Requires:* The `order` argument shall not be `memory_order_release` nor `memory_order_acq_rel`.

2 *Effects:* Repeatedly performs the following steps, in order:

(2.1) — Evaluates `object->load(order) != old` then, if the result is `true`, returns.

(2.2) — Blocks until an implementation-defined condition has been met. [*Note:* Consequently, it may unblock for reasons other than an atomic notifying operation. — *end note*]

3 *Remarks:* This function is an atomic waiting operation.

```
void notify_one() volatile noexcept;
void notify_one() noexcept;
```

4 *Effects:* Unblocks up to execution of an atomic waiting operation that blocked after observing the result of an atomic operation `X`, if there exists another atomic operation `Y`, such that `X` precedes `Y` in the modification order of `*this`, and `Y` happens before this call.

5 *Remarks:* This function is an atomic notifying operation.

```
void notify_all() volatile noexcept;
void notify_all() noexcept;
```

6 *Effects:* Unblocks each execution of an atomic waiting operation that blocked after observing the result of an atomic operation `X`, if there exists another atomic operation `Y`, such that `X` precedes `Y` in the modification order of `*this`, and `Y` happens before this call.

7 *Remarks:* This function is an atomic notifying operation.

Modify [**atomics.types.int**] paragraph 1 as follows:

30.7.2 Specializations for integers [atomics.types.int]

1 There are specializations of the `atomic` class template for the integral types `char`, `signed char`, `unsigned char`, `short`, `unsigned short`, `int`, `unsigned int`, `long`, `unsigned long`, `long long`, `unsigned long long`, `char8_t`, `char16_t`, `char32_t`, `wchar_t`, and any other types needed by the typedefs in the header `<cstdint>`. For each such type *`integral`*, the specialization `atomic<`*`integral`*`>` provides additional atomic operations appropriate to integral types. [*Note:* For the specialization `atomic<bool>`, see 30.7. — *end note*]

```
namespace std {
  template<> struct atomic<integral> {
    using value_type = integral;
    using difference_type = value_type;
    static constexpr bool is_always_lock_free = implementation-defined;
    bool is_lock_free() const volatile noexcept;
    bool is_lock_free() const noexcept;
    void store(integral, memory_order = memory_order::seq_cst) volatile no
    void store(integral, memory_order = memory_order::seq_cst) noexcept;
    integral load(memory_order = memory_order::seq_cst) const volatile noe
    integral load(memory_order = memory_order::seq_cst) const noexcept;
    operator integral() const volatile noexcept;
    operator integral() const noexcept;
    integral exchange(integral, memory_order = memory_order::seq_cst) vola
    integral exchange(integral, memory_order = memory_order::seq_cst) noex
    bool compare_exchange_weak(integral&, integral,
                               memory_order, memory_order) volatile noexce
    bool compare_exchange_weak(integral&, integral,
                               memory_order, memory_order) noexcept;
    bool compare_exchange_strong(integral&, integral,
                                 memory_order, memory_order) volatile noex
    bool compare_exchange_strong(integral&, integral,
                                 memory_order, memory_order) noexcept;
    bool compare_exchange_weak(integral&, integral,
                               memory_order = memory_order::seq_cst) volat
    bool compare_exchange_weak(integral&, integral,
                               memory_order = memory_order::seq_cst) noexc
    bool compare_exchange_strong(integral&, integral,
                                 memory_order = memory_order::seq_cst) vol
    bool compare_exchange_strong(integral&, integral,
                                 memory_order = memory_order::seq_cst) noe
```

```
    void wait(integral old, memory_order = memory_order::seq_cst) const vo
    void wait(integral old, memory_order = memory_order::seq_cst) const no
    void notify_one() volatile noexcept;
    void notify_one() noexcept;
    void notify_all() volatile noexcept;
    void notify_all() noexcept;
```

```
    integral fetch_add(integral, memory_order = memory_order::seq_cst) vol
    integral fetch_add(integral, memory_order = memory_order::seq_cst) noe
    integral fetch_sub(integral, memory_order = memory_order::seq_cst) vol
    integral fetch_sub(integral, memory_order = memory_order::seq_cst) noe
    integral fetch_and(integral, memory_order = memory_order::seq_cst) vol
    integral fetch_and(integral, memory_order = memory_order::seq_cst) noe
    integral fetch_or(integral, memory_order = memory_order::seq_cst) vola
    integral fetch_or(integral, memory_order = memory_order::seq_cst) noex
    integral fetch_xor(integral, memory_order = memory_order::seq_cst) vol
    integral fetch_xor(integral, memory_order = memory_order::seq_cst) noe

    atomic() noexcept = default;
    constexpr atomic(integral) noexcept;
    atomic(const atomic&) = delete;
    atomic& operator=(const atomic&) = delete;
    atomic& operator=(const atomic&) volatile = delete;
    integral operator=(integral) volatile noexcept;
    integral operator=(integral) noexcept;

    integral operator++(int) volatile noexcept;
    integral operator++(int) noexcept;
    integral operator--(int) volatile noexcept;
    integral operator--(int) noexcept;
    integral operator++() volatile noexcept;
    integral operator++() noexcept;
    integral operator--() volatile noexcept;
    integral operator--() noexcept;
    integral operator+=(integral) volatile noexcept;
    integral operator+=(integral) noexcept;
    integral operator-=(integral) volatile noexcept;
    integral operator-=(integral) noexcept;
    integral operator&=(integral) volatile noexcept;
    integral operator&=(integral) noexcept;
    integral operator|=(integral) volatile noexcept;
    integral operator|=(integral) noexcept;
    integral operator^=(integral) volatile noexcept;
    integral operator^=(integral) noexcept;
  };
}
```

Modify [**atomics.types.float**] paragraph 1 as follows:

30.7.3 Specializations for floating-point types [atomics.types.float]

1 There are specializations of the `atomic` class template for the floating-point types `float`, `double`, and `long double`. For each such type *`floating-point`*, the specialization `atomic<`*`floating-point`*`>` provides additional atomic operations appropriate to floating-point types.

```
namespace std {
  template<> struct atomic<floating-point> {
    using value_type = floating-point;
    using difference_type = value_type;
    static constexpr bool is_always_lock_free = implementation-defined;
    bool is_lock_free() const volatile noexcept;
    bool is_lock_free() const noexcept;
    void store(floating-point, memory_order = memory_order::seq_cst) volat
    void store(floating-point, memory_order = memory_order::seq_cst) noexc
    floating-point load(memory_order = memory_order::seq_cst) const volati
    floating-point load(memory_order = memory_order::seq_cst) const noexce
    operator floating-point() const volatile noexcept;
    operator floating-point() const noexcept;
    floating-point exchange(floating-point,
                            memory_order = memory_order::seq_cst) volatil
    floating-point exchange(floating-point,
                            memory_order = memory_order::seq_cst) noexcep
    bool compare_exchange_weak(floating-point&, floating-point,
                               memory_order, memory_order) volatile noexce
    bool compare_exchange_weak(floating-point&, floating-point,
                               memory_order, memory_order) noexcept;
    bool compare_exchange_strong(floating-point&, floating-point,
                                 memory_order, memory_order) volatile noex
    bool compare_exchange_strong(floating-point&, floating-point,
                                 memory_order, memory_order) noexcept;
    bool compare_exchange_weak(floating-point&, floating-point,
                               memory_order = memory_order::seq_cst) volat
    bool compare_exchange_weak(floating-point&, floating-point,
                               memory_order = memory_order::seq_cst) noexc
    bool compare_exchange_strong(floating-point&, floating-point,
                                 memory_order = memory_order::seq_cst) vol
    bool compare_exchange_strong(floating-point&, floating-point,
                                 memory_order = memory_order::seq_cst) noe

    void wait(floating-point old, memory_order = memory_order::seq_cst) co
```

```
    void wait(floating-point old, memory_order = memory_order::seq_cst) co
    void notify_one() volatile noexcept;
    void notify_one() noexcept;
    void notify_all() volatile noexcept;
    void notify_all() noexcept;

    floating-point fetch_add(floating-point,
                                    memory_order = memory_order::seq_cst)
    floating-point fetch_add(floating-point,
                                    memory_order = memory_order::seq_cst)
    floating-point fetch_sub(floating-point,
                                    memory_order = memory_order::seq_cst)
    floating-point fetch_sub(floating-point,
                                    memory_order = memory_order::seq_cst)

    atomic() noexcept = default;
    constexpr atomic(floating-point) noexcept;
    atomic(const atomic&) = delete;
    atomic& operator=(const atomic&) = delete;
    atomic& operator=(const atomic&) volatile = delete;
    floating-point operator=(floating-point) volatile noexcept;
    floating-point operator=(floating-point) noexcept;

    floating-point operator+=(floating-point) volatile noexcept;
    floating-point operator+=(floating-point) noexcept;
    floating-point operator-=(floating-point) volatile noexcept;
    floating-point operator-=(floating-point) noexcept;
  };
}
```

Modify [**atomics.types.pointer**] paragraph 1 as follows:

30.7.4 Partial specialization for pointers [atomics.types.pointer]

```
namespace std {
  template<class T> struct atomic<T*> {
    using value_type = T*;
    using difference_type = ptrdiff_t;
    static constexpr bool is_always_lock_free = implementation-defined;
    bool is_lock_free() const volatile noexcept;
    bool is_lock_free() const noexcept;
    void store(T*, memory_order = memory_order::seq_cst) volatile noexcept
    void store(T*, memory_order = memory_order::seq_cst) noexcept;
    T* load(memory_order = memory_order::seq_cst) const volatile noexcept;
    T* load(memory_order = memory_order::seq_cst) const noexcept;
    operator T*() const volatile noexcept;
    operator T*() const noexcept;
    T* exchange(T*, memory_order = memory_order::seq_cst) volatile noexcep
    T* exchange(T*, memory_order = memory_order::seq_cst) noexcept;
    bool compare_exchange_weak(T*&, T*, memory_order, memory_order) volati
    bool compare_exchange_weak(T*&, T*, memory_order, memory_order) noexce
    bool compare_exchange_strong(T*&, T*, memory_order, memory_order) noex
    bool compare_exchange_weak(T*&, T*,
                               memory_order = memory_order::seq_cst) volat
    bool compare_exchange_weak(T*&, T*,
                               memory_order = memory_order::seq_cst) noexc
    bool compare_exchange_strong(T*&, T*,
                                 memory_order = memory_order::seq_cst) vol
    bool compare_exchange_strong(T*&, T*,
                                 memory_order = memory_order::seq_cst) noe
    void wait(T* old, memory_order = memory_order::seq_cst) const volatile
    void wait(T* old, memory_order = memory_order::seq_cst) const noexcept
    void notify_one() volatile noexcept;
    void notify_one() noexcept;
    void notify_all() volatile noexcept;
    void notify_all() noexcept;
```

```
    T* fetch_add(ptrdiff_t, memory_order = memory_order::seq_cst) volatile
    T* fetch_add(ptrdiff_t, memory_order = memory_order::seq_cst) noexcept
    T* fetch_sub(ptrdiff_t, memory_order = memory_order::seq_cst) volatile
    T* fetch_sub(ptrdiff_t, memory_order = memory_order::seq_cst) noexcept

    atomic() noexcept = default;
    constexpr atomic(T*) noexcept;
    atomic(const atomic&) = delete;
    atomic& operator=(const atomic&) = delete;
    atomic& operator=(const atomic&) volatile = delete;
    T* operator=(T*) volatile noexcept;
    T* operator=(T*) noexcept;

    T* operator++(int) volatile noexcept;
    T* operator++(int) noexcept;
    T* operator--(int) volatile noexcept;
    T* operator--(int) noexcept;
    T* operator++() volatile noexcept;
    T* operator++() noexcept;
    T* operator--() volatile noexcept;
    T* operator--() noexcept;
    T* operator+=(ptrdiff_t) volatile noexcept;
    T* operator+=(ptrdiff_t) noexcept;
    T* operator-=(ptrdiff_t) volatile noexcept;
    T* operator-=(ptrdiff_t) noexcept;
  };
}
```

1 There is a partial specialization of the `atomic` class template for pointers. Specializations of this partial specialization are standard-layout structs. They each have a trivial default constructor and a trivial destructor.

Modify [**atomics.flag**] as follows:

30.9 Flag type and operations **[atomics.flag]**

```
namespace std {
  struct atomic_flag {
    bool test(memory_order = memory_order_seq_cst) volatile noexcept;
    bool test(memory_order = memory_order_seq_cst) noexcept;
    bool test_and_set(memory_order = memory_order_seq_cst) volatile noexce
    bool test_and_set(memory_order = memory_order_seq_cst) noexcept;
    void clear(memory_order = memory_order_seq_cst) volatile noexcept;
    void clear(memory_order = memory_order_seq_cst) noexcept;

    void wait(bool, memory_order = memory_order::seq_cst) const volatile n

    void wait(bool, memory_order = memory_order::seq_cst) const noexcept;
    void notify_one() volatile noexcept;
    void notify_one() noexcept;
    void notify_all() volatile noexcept;
    void notify_all() noexcept;

    atomic_flag() noexcept = default;
    atomic_flag(const atomic_flag&) = delete;
    atomic_flag& operator=(const atomic_flag&) = delete;
    atomic_flag& operator=(const atomic_flag&) volatile = delete;
  };

  bool atomic_flag_test(volatile atomic_flag*) noexcept;
  bool atomic_flag_test(atomic_flag*) noexcept;
  bool atomic_flag_test_explicit(volatile atomic_flag*, memory_order) noex
  bool atomic_flag_test_explicit(atomic_flag*, memory_order) noexcept;

  bool atomic_flag_test_and_set(volatile atomic_flag*) noexcept;
  bool atomic_flag_test_and_set(atomic_flag*) noexcept;
  bool atomic_flag_test_and_set_explicit(volatile atomic_flag*, memory_orc
  bool atomic_flag_test_and_set_explicit(atomic_flag*, memory_order) noexc
  void atomic_flag_clear(volatile atomic_flag*) noexcept;
  void atomic_flag_clear(atomic_flag*) noexcept;
  void atomic_flag_clear_explicit(volatile atomic_flag*, memory_order) noe
  void atomic_flag_clear_explicit(atomic_flag*, memory_order) noexcept;
```

```
  void atomic_flag_wait(const volatile atomic_flag*, bool) noexcept;
  void atomic_flag_wait(const atomic_flag*, bool) noexcept;
  void atomic_flag_wait_explicit(const volatile atomic_flag*, bool, memory
  void atomic_flag_wait_explicit(const atomic_flag*, bool, memory_order) r
  void atomic_flag_notify_one(volatile atomic_flag*) noexcept;
  void atomic_flag_notify_one(atomic_flag*) noexcept;
  void atomic_flag_notify_all(volatile atomic_flag*) const noexcept;
  void atomic_flag_notify_all(atomic_flag*) const noexcept;
```

```
  #define ATOMIC_FLAG_INIT see below
}
```

1 The `atomic_flag` type provides the classic test-and-set functionality. It has two states, set and clear.

2 Operations on an object of type `atomic_flag` shall be lock-free. [*Note:* Hence the operations should also be address-free. — *end note*]

3 The `atomic_flag` type is a standard-layout struct. It has a trivial default constructor and a trivial destructor.

4 The macro `ATOMIC_FLAG_INIT` shall be defined in such a way that it can be used to initialize an object of type `atomic_flag` to the clear state. The macro can be used in the form:

```
atomic_flag guard = ATOMIC_FLAG_INIT;
```

It is unspecified whether the macro can be used in other initialization contexts. For a complete static-duration object, that initialization shall be static. Unless initialized with `ATOMIC_FLAG_INIT`, it is unspecified whether an `atomic_flag` object has an initial state of set or clear.

```
bool atomic_flag_test(volatile atomic_flag* object) noexcept;
bool atomic_flag_test(atomic_flag* object) noexcept;
bool atomic_flag_test_explicit(volatile atomic_flag* object, memory_order
bool atomic_flag_test_explicit(atomic_flag* object, memory_order order) no
bool atomic_flag::test(memory_order order = memory_order_seq_cst) volatile
bool atomic_flag::test(memory_order order = memory_order_seq_cst) noexcept
```

- *Requires:* The `order` argument shall not be `memory_order_release` nor `memory_order_acq_rel`.
- *Effects:* Memory is affected according to the value of `order`.

— *Returns:* Atomically returns the value pointed to by `object` or `this`.

```
bool atomic_flag_test_and_set(volatile atomic_flag* object) noexcept;
bool atomic_flag_test_and_set(atomic_flag* object) noexcept;
bool atomic_flag_test_and_set_explicit(volatile atomic_flag* object,
                                       memory_order order) noexcept;
bool atomic_flag_test_and_set_explicit(atomic_flag* object, memory_order o
bool atomic_flag::test_and_set(memory_order order = memory_order_seq_cst)
bool atomic_flag::test_and_set(memory_order order = memory_order_seq_cst)
```

5 *Effects:* Atomically sets the value pointed to by `object` or by `this` to `true`. Memory is affected according to the value of `order`. These operations are atomic read-modify-write operations (4.7).

6 *Returns:* Atomically, the value of the object immediately before the effects.

```
void atomic_flag_clear(volatile atomic_flag* object) noexcept;
void atomic_flag_clear(atomic_flag* object) noexcept;
void atomic_flag_clear_explicit(volatile atomic_flag* object,
                                memory_order order) noexcept;
void atomic_flag_clear_explicit(atomic_flag* object, memory_order order) n
void atomic_flag::clear(memory_order order = memory_order_seq_cst) volatil
void atomic_flag::clear(memory_order order = memory_order_seq_cst) noexcep
```

Requires: The `order` argument shall not be `memory_order_consume`, `memory_order_acquire`, nor `memory_order_acq_rel`.

Effects: Atomically sets the value pointed to by `object` or by `this` to `false`. Memory is affected according to the value of `order`.

```
void atomic_flag_wait(const volatile atomic_flag* object, bool old) noexce
void atomic_flag_wait(const atomic_flag* object, bool old) noexcept;
void atomic_flag_wait_explicit(const volatile atomic_flag* object,
                               bool old, memory_order order) noexcept;
void atomic_flag_wait_explicit(const atomic_flag* object,
                               bool old, memory_order order) noexcept;
void atomic_flag::wait(bool old,
                       memory_order order = memory_order::seq_cst) const v
void atomic_flag::wait(bool old,
                       memory_order order = memory_order::seq_cst) const n
```

7 *Requires:* The `order` argument shall not be `memory_order_release` nor `memory_order_acq_rel`.

8 *Effects:* Repeatedly performs the following steps, in order:

(8.1) — Evaluates `object->load(order) != old` then, if the result is `true`, returns.

(8.2) — Blocks until an implementation-defined condition has been met. [*Note:* Consequently, it may unblock for reasons other than an atomic notifying operation. — *end note*]

9 *Remarks:* This function is an atomic waiting operation.

```
void atomic_flag_notify_one(volatile atomic_flag* object) noexcept;
void atomic_flag_notify_one(atomic_flag* object) noexcept;
void atomic_flag::notify_one() volatile noexcept;
void atomic_flag::notify_one() noexcept;
```

10 *Effects:* Unblocks up to one execution of a atomic waiting operation that blocked after observing the result of an atomic operation `X`, if there exists another atomic operation `Y`, such that `X` precedes `Y` in the modification order of `*object` or `*this`, and `Y` happens before this call.

11 *Remarks:* This function is an atomic notifying operation.

```
void atomic_flag_notify_all(volatile atomic_flag* object) const noexcept;
void atomic_flag_notify_all(atomic_flag* object) const noexcept;
void atomic_flag::notify_all() volatile noexcept;
void atomic_flag::notify_all() noexcept;
```

12 *Effects:* Unblocks each execution of a atomic waiting operation that blocked after observing the result of an atomic operation `X`, if there exists another atomic operation `Y`, such that `X` precedes `Y` in the modification order of `*object` or `*this`, and `Y` happens before this call.

13 *Remarks:* This function is an atomic notifying operation.

Modify Table 134 "Thread support library summary" in **[thread.general]** as follows:

Table 134 — Thread support library summary

	Subclause	Header(s)
31.2	Requirements	
31.3	Threads	`<thread>`
31.4	Mutual exclusion	`<mutex>` `<shared_mutex>`
31.5	Condition variables	`<condition_variable>`
31.□	Semaphores	`<semaphore>`
31.□	Latches and barriers	`<latch>` `<barrier>`
31.6	Futures	`<future>`

Add two new subclauses after **[thread.condition]**:

31.□ Semaphores [thread.semaphore]

1 Semaphores are lightweight synchronization primitives used to constrain concurrent access to a shared resource. They are widely used to implement other synchronization primitives and, whenever both are applicable, can be more efficient than condition variables.

2 A counting semaphore is a semaphore object that models a non-negative resource count. A binary semaphore is a semaphore object that has only two states, also known as available and unavailable. [*Note:* A binary semaphore should be more efficient than a counting semaphore with a unit magnitude count. – *end note*]

31.□.1 Header `<semaphore>` synopsis [thread.semaphore.syn]

```
namespace std {
  template<ptrdiff_t least_max_value = implementation-defined>
    class counting_semaphore;

  using binary_semaphore = counting_semaphore<1>;
}
```

31.□.2 Class template `counting_semaphore` [thread.semaphore.counting.class]

```
namespace std {
  template<ptrdiff_t least_max_value>
  class counting_semaphore {
  public:
    static constexpr ptrdiff_t max() noexcept;

    constexpr explicit counting_semaphore(ptrdiff_t desired);
    ~counting_semaphore();

    counting_semaphore(const basic_semaphore&) = delete;
    counting_semaphore(basic_semaphore&&) = delete;
    counting_semaphore& operator=(const basic_semaphore&) = delete;
    counting_semaphore& operator=(basic_semaphore&&) = delete;

    void release(ptrdiff_t update = 1);
    void acquire();
    bool try_acquire() noexcept;
    template<class Rep, class Period>
      bool try_acquire_for(const chrono::duration<Rep, Period>& rel_time);
    template<class Clock, class Duration>
      bool try_acquire_until(const chrono::time_point<Clock, Duration>& ab

  private:
    ptrdiff_t counter; // exposition only
  };
}
```

1 Class `counting_semaphore` maintains an internal counter that is initialized when the semaphore is created. Threads may block waiting until `counter >= 1`.

2 Semaphores permit concurrent invocation of the `release`, `acquire`, `try_acquire`, `try_acquire_for`, and `try_acquire_until` member functions.

```
static constexpr ptrdiff_t max() noexcept;
```

3 *Returns:* The maximum value of `counter`. This value shall not be less than that of the template argument `least_max_value`. [*Note:* The value may exceed least_max_value. – *end note*]

```
constexpr explicit counting_semaphore(ptrdiff_t desired);
```

4 *Requires:* `desired >= 0` and `desired <= max()`.

5 *Effects:* `counter = desired`.

6 *Throws:* Nothing.

`~counting_semaphore();`

7 *Requires:* For every function call that blocks on `counter`, a function call that will cause it to unblock and return shall happen before this call. [*Note:* This relaxes the usual rules, which would have required all wait calls to happen before destruction. — *end note*]

8 *Effects:* Destroys the object.

9 *Throws:* Nothing.

`void release(ptrdiff_t update = 1);`

10 *Requires:* `update >= 0`, and `counter + update <= max()`.

11 *Effects:* `counter += update`, executed atomically. If any threads are blocked on counter, unblocks them.

12 *Synchronization:* Strongly happens before invocations of `try_acquire` that observe the result of the effects.

13 *Throws:* `system_error` when an exception is required (31.2.2).

`bool try_acquire() noexcept;`

14 *Effects:*

(14.1) — With low probability, returns immediately. [*Note:* An implementation should ensure that `try_acquire` does not consistently return `false` in the absence of contending acquisitions. — *end note*]

(14.2) — Otherwise, if `counter >= 1`, then `counter -= 1` is executed atomically.

15 *Returns:* `true` if `counter` was decremented, otherwise `false`.

`void acquire();`

16 *Effects:* Repeatedly performs the following steps, in order:

(16.1) — Evaluates `try_acquire`, then, if the result is `true`, returns.

(16.2) — Blocks until `counter >= 1`.

17 *Throws:* `system_error` when an exception is required (31.2.2).

```
template<class Rep, class Period>
  bool try_acquire_for(const chrono::duration<Rep, Period>& rel_time);
template<class Clock, class Duration>
  bool try_acquire_until(const chrono::time_point<Clock, Duration>& abs_ti
```

18 *Effects:* Repeatedly performs the following steps, in order:

(18.1) — Evaluates `try_acquire`. If the result is `true`, returns `true`.

(18.2) — Blocks until the timeout expires or `counter >= 1`. If the timeout expired, returns `false`.

19 *Throws:* Timeout-related exceptions (31.2.4).

31.□ Coordination Types [thread.coord]

1 This section describes various concepts related to thread coordination, and defines the ***coordination types*** `latch` and `barrier`. These types facilitate concurrent computation performed by a number of threads, in one or more ***phases***.

2 In this subclause, a synchronization point represents a condition that a thread may contribute to or wait for, potentially blocking until it is satisfied. A thread arrives at the synchronization point when it has an effect on the state of the condition, even if it does not cause it to become satisfied.

3 Concurrent invocations of the member functions of coordination types, other than their destructors, do not introduce data races.

31.□.1 Latches [thread.coord.latch]

1 A latch is a thread coordination mechanism that allows any number of threads to block until an expected count is summed (exactly) by threads that arrived at the latch. The expected count is set when the latch is constructed. An individual latch is a single-use object; once the count has been reached, the latch cannot be reused.

31.□.1.1 Header `<latch>` synopsis [thread.coord.latch.syn]

```
namespace std {
  class latch;
}
```

31.□.1.2 Class `latch` [thread.coord.latch.class]

```
namespace std {
  class latch {
  public:
    constexpr explicit latch(ptrdiff_t expected);
    ~latch();

    latch(const latch&) = delete;
    latch(latch&&) = delete;
    latch& operator=(const latch&) = delete;
    latch& operator=(latch&&) = delete;

    void count_down(ptrdiff_t update = 1);
    bool try_wait() const noexcept;
    void wait() const;
    void arrive_and_wait(ptrdiff_t update = 1);

  private:
    ptrdiff_t counter; // exposition only
  };
}
```

1 A `latch` maintains an internal counter that is initialized when the `latch` is created. Threads may block at the `latch`'s synchronization point, waiting for `counter` to be decremented to `0`.

```
constexpr explicit latch(ptrdiff_t expected);
```

2 *Requires:* `expected >= 0`.

3 *Effects:* `counter = expected`.

4 *Throws:* Nothing.

```
~latch();
```

5 *Requires:* No threads are blocked at the synchronization point.

6 *Effects:* Destroys the latch.

7 *Throws:* Nothing.

8 *Remarks:* May be called even if some threads have not yet returned from functions that block at the synchronization point, provided that they are unblocked. [*Note:* The destructor may

block until all threads have exited invocations of `wait` on this object. — *end note*]

```
void count_down(ptrdiff_t update = 1);
```

9 *Requires:* `counter >= update` and `update >= 0`.

10 *Effects:* Atomically decrements `counter` by `update`.

11 *Synchronization:* Synchronizes with the returns from all calls unblocked by the effects.

12 *Throws:* `system_error` when an exception is required (31.2.2).

13 *Remarks:* Arrives at the synchronization point with `update` count.

```
bool try_wait() const noexcept;
```

14 *Returns:* `counter == 0`.

```
void wait() const noexcept;
```

15 *Effects:* If `counter == 0`, returns immediately. Otherwise, blocks the calling thread at the synchronization point until `counter == 0`.

16 *Throws:* `system_error` when an exception is required (31.2.2).

```
void arrive_and_wait(ptrdiff_t update = 1);
```

17 *Effects:* Equivalent to `count_down(update); wait();`.

18 *Throws:* `system_error` when an exception is required (31.2.2).

31.□.2 Barriers [thread.coord.barrier]

1 A barrier is a thread coordination mechanism that allows at most an expected count of threads to block until that count is summed (exactly) by threads that arrived at the barrier in each of its successive phases. Once threads are released from blocking at the synchronization point for a phase, they can reuse the same barrier immediately in its next phase. [*Note:* It is thus useful for managing repeated tasks, or phases of a larger task, that are handled by multiple threads. — *end note*]

2 A barrier has a ***completion step*** that is a (possibly empty) set of effects associated with a phase of the barrier. When the member functions defined in this subclause arrive at the barrier, they have the following effects:

(2.1) — When the expected number of threads for this phase have arrived at the barrier, one of those threads executes the barrier type's completion step.

(2.2) — When the completion step is completed, all threads blocked at the synchronization point for this phase are unblocked and the barrier enters its next phase. The end of the completion step strongly happens before the returns from all calls unblocked by its completion.

31.□.2.1 Header <barrier> synopsis [thread.coord.barrier.syn]

```
namespace std {
  template<class CompletionFunction = implementation-defined>
    class barrier;
}
```

31.□.2.2 Class template `barrier` [thread.coord.barrier.class]

```
namespace std {
  template<class CompletionFunction>
  class barrier {
  public:
    using arrival_token = implementation-defined;

    constexpr explicit barrier(ptrdiff_t expected,
                               CompletionFunction f = CompletionFunction());

    ~barrier();

    barrier(const barrier&) = delete;
    barrier(barrier&&) = delete;
    barrier& operator=(const barrier&) = delete;
    barrier& operator=(barrier&&) = delete;

    [[nodiscard]] arrival_token arrive(ptrdiff_t update = 1);
    void wait(arrival_token&& arrival) const;

    void arrive_and_wait();
    void arrive_and_drop();

  private:
    CompletionFunction completion; // exposition only
  };
}
```

1 A `barrier` is a barrier type with a completion step controlled by a function object. The completion step calls `completion`. Threads may block at the barrier's synchronization point for a phase, waiting for the expected sum contributions by threads that arrive in that phase.

2 `CompletionFunction` shall be `Cpp17MoveConstructible` (Table 26), `is_invocable_r_v<void, CompletionFunction>` shall be `true`, and `noexcept(declval<CompletionFunction>()())` shall be `true`.

3 `barrier::arrival_token` is an implementation-defined type. `is_nothrow_move_constructible_v<barrier::arrival_token>` shall be `true` and `is_nothrow_move_assignable_v<barrier::arrival_token>` shall be `true`.

```
constexpr explicit barrier(ptrdiff_t expected,
                           CompletionFunction f = CompletionFunction());
```

4 *Requires:* `expected >= 0`, and `noexcept(f())` shall be `true`.

5 *Effects:* Initializes the barrier for `expected` number of threads in the first phase, and initializes `completion` with `move(f)`. [*Note:* If `expected` is `0` this object may only be destroyed. — *end note*]

6 *Throws:* Any exception thrown by `CompletionFunction`'s move constructor.

```
~barrier();
```

7 *Requires:* No threads are blocked at a synchronization point for any phase.

8 *Effects:* Destroys the barrier.

9 *Remarks:* May be called even if some threads have not yet returned from functions that block at a synchronization point, provided that they have unblocked. [*Note:* The destructor may block until all threads have exited invocations of wait() on this object. — *end note*]

```
[[nodiscard]] arrival_token arrive(ptrdiff_t update = 1);
```

10 *Requires:* The expected count is not less than `update`.

11 *Effects:* Constructs an object of type `arrival_token` that is associated with the `barrier`'s synchronization point for the current phase, then arrives `update` times at the synchronization point for the current phase.

12 *Synchronization:* The call to `arrive` strongly happens before the start of the completion step for the current phase.

13 *Returns:* The constructed object.

14 *Throws:* `system_error` when an exception is required (31.2.2).

15 *Remarks:* This may cause the completion step to start.

```
void wait(arrival_token&& arrival) const;
```

16 *Requires:* `arrival` is associated with a synchronization point for the current or the immediately preceding phases of the `barrier`.

17 *Effects:* Blocks at the synchronization point associated with `std::move(arrival)` until the condition is satisfied.

18 *Throws:* `system_error` when an exception is required (31.2.2).

```
void arrive_and_wait();
```

19 *Effects:* Equivalent to `wait(arrive())`.

20 *Throws:* `system_error` when an exception is required (31.2.2).

```
void arrive_and_drop();
```

21 *Requires:* The expected number of threads for the current phase is not `0`.

22 *Effects:* Decrements the expected number of threads for subsequent phases by `1`, then arrives at the synchronization point for the current phase.

23 *Synchronization:* The call to `arrive_and_drop` strongly happens before the start of the completion step for the current phase.

24 *Throws:* `system_error` when an exception is required (31.2.2).

25 *Remarks:* This may cause the completion step to start.

Create the following feature test macros:

— `__cpp_lib_atomic_lock_free_type_aliases`, which implies that `atomic_signed_lock_free` and `atomic_unsigned_lock_free` types are available.

— `__cpp_lib_atomic_flag_test`, which implies the `test` methods and free functions for `atomic_flag` are available.

— `__cpp_lib_atomic_wait`, which implies the `notify_*` and `wait` methods and free functions for `atomic` and `atomic_flag` are available.

— `__cpp_lib_semaphore`, which implies that `counting_semaphore` and `binary_semaphore` are available.

— `__cpp_lib_latch`, which implies that `latch` is available.

— `__cpp_lib_barrier`, which implies that `barrier` is available.

§ Index

§ Terms defined by this specification

§ References

§ Informative References

[N4791]

Richard Smith. Working Draft, Standard for Programming Language C++. 26 November 2018. URL: https://wg21.link/n4791

[P0514R4]

Olivier Giroux. Efficient concurrent waiting for C++20. 3 May 2018. URL: https://wg21.link/p0514r4

[P0666R2]

Olivier Giroux. Revised Latches and Barriers for C++20. 6 May 2018. URL: https://wg21.link/p0666r2

[P0995R1]

JF Bastien, Olivier Giroux, Andrew Hunter. Improving atomic_flag. 22 June 2018. URL: https://wg21.link/p0995r1

[P1258R0]

Detlef Vollmann. Don't Make C++ Unimplementable On Small CPUs. 8 October 2018. URL: https://wg21.link/p1258r0